Machine learning
- Optimization

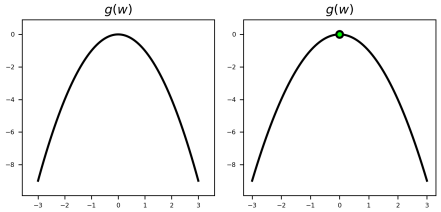
CdL Informatica e Innovazione Digitale

Valerio Freschi

Università di Urbino - DiSPeA
valerio.freschi@uniurb.it

Mathematical optimization

- The problem: determining global minimum/maximum value a function can take
- A centuries old problem with applications in sciences and engineering
- Example:



Mathematical optimization

- Given a function of N variables $g(w_1, w_2, ...w_N)$, we aim at solving the following minimization problem:
- $\underset{\boldsymbol{w}}{\text{minimize}} \;\; g(\boldsymbol{w})$
- We aim at finding $\boldsymbol{w}^\star$: $g(\boldsymbol{w}^\star) \leq g(\boldsymbol{w})$ for all $\boldsymbol{w}$.
- Maximization can be expressed in terms of a minimization problem as:
  - $\underset{\boldsymbol{w}}{\text{minimize}} \;\; -g(\boldsymbol{w})$
- Definition of *global* maxima/minima can be extended to *local* maxima/minima

Mathematical optimization

The zero-order optimality condition

- A point $\boldsymbol{w}^\star$ is
  - a global minimum of $g(\boldsymbol{w})$ if and only if $g(\boldsymbol{w}^\star) \leq g(\boldsymbol{w})$ for all $\boldsymbol{w}$
  - a global maximum of $g(\boldsymbol{w})$ if and only if $g(\boldsymbol{w}^\star) \geq g(\boldsymbol{w})$ for all $\boldsymbol{w}$
  - a local minimum of $g(\boldsymbol{w})$ if and only if $g(\boldsymbol{w}^\star) \leq g(\boldsymbol{w})$ for all $\boldsymbol{w}$ near $\boldsymbol{w}^\star$
  - a local maximum of $g(\boldsymbol{w})$ if and only if $g(\boldsymbol{w}^\star) \geq g(\boldsymbol{w})$for all $\boldsymbol{w}$ near $\boldsymbol{w}^\star$

- The condition is called zero-order because optima are defined in terms of the function itself
- Higher order conditions involve the use of first and second derivatives of the function (1st-order and 2nd-order optimality conditions)

Global optimization

- Global optimisation entails:
  - evaluating the function at a large number of input points
  - choosing the lowest value to approximate the global minimum
- How can we choose points? (∞ even for a single-input continuous function)
- Uniform sampling
- Random sampling
- Both can be seen as an approximation of 0th-order optimality condition
- Examples:
  - $g(w) = w^2 + 0.2$, global minimum at $w = 0$
  - $g(w_1, w_2) = w_1^2 + w_2^2 + 0.2$, global minimum at $\boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Global optimization

- Examples:

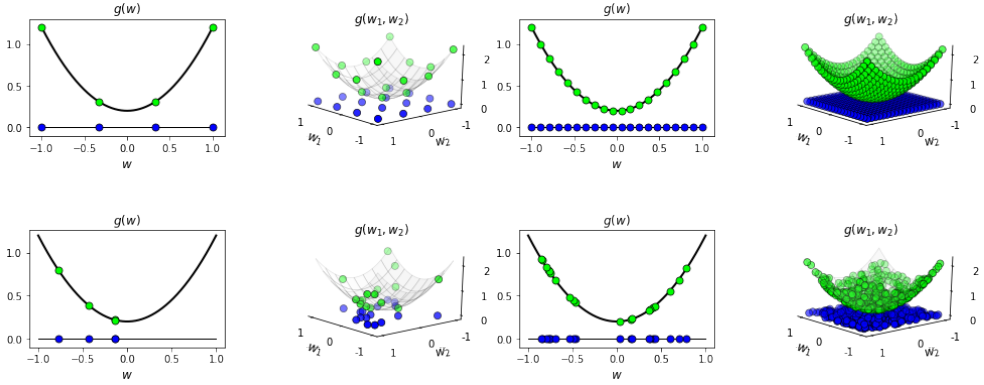
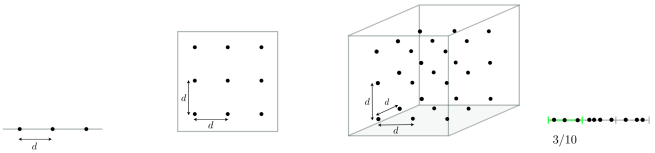


Figure: Global minimization by sparse (a) and dense (b) sampling.

The curse of dimensionality

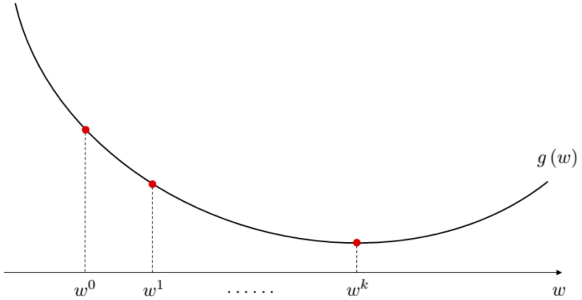
- Global optimisation based on zero-order evaluations only works for low-dimensional functions
- Hence it is almost unusable in modern machine learning (input dimensions from ~ 100, to $\sim 10^5$, up to $\sim 10^6$)



(a) (b)

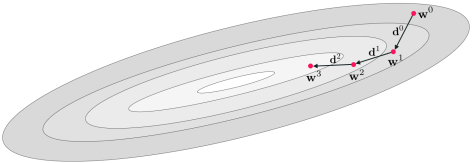
Local optimization

- Local optimization by sequential minimization of function in steps
- Lots of approaches, but a common framework
- Starts from an initial point
- Sequentially proceeds pulling it *downhill*



Local optimization

- The general approach:
- Start from initial point $\boldsymbol{w}^0$
- Find a *descent direction*, i.e. a direction vector $\boldsymbol{d}^0$ beginning at $\boldsymbol{w}^0$ and pointing towards a new point $\boldsymbol{w}^1$
- $\boldsymbol{w}^1$ has a lower function evaluation
- Find a new descent direction $\boldsymbol{d}^1$ moving downhill from $\boldsymbol{w}^1$ and a new point $\boldsymbol{w}^2$
- Repeat the process: K points are computed in K steps



Local optimization

- A step *length parameter* (a.k.a. *learning rate*) is introduced
- The generic step is computed as: $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}^{k-1}$
- The descent direction $\boldsymbol{d}^{k-1}$ is scaled by the steplength parameter α
- Hence, the distance covered at the kth step of the local optimization scheme results to be: $\left\| \boldsymbol{w}^k - \boldsymbol{w}^{k-1} \right\|_2 = \left\| \left(\boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}^{k-1} \right) - \boldsymbol{w}^{k-1} \right\|_2 = \alpha \left\| \boldsymbol{d}^{k-1} \right\|_2$
- α allows us to tune the amount of distance covered in the descent vector direction
- How to choose α? Usually some fixed, small value. But not only... (stay tuned)

Random search

- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?

Random search

- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?
  - At step K sample a given number (say P) of directions from $\boldsymbol{w}^{k-1}$

Random search

- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?
  - At step K sample a given number (say P) of directions from $\boldsymbol{w}^{k-1}$
  - Each direction $\boldsymbol{d}^p$ defines a candidate point $\boldsymbol{w}^{k-1} + \boldsymbol{d}^p$

Random search

- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?
  - At step K sample a given number (say P) of directions from $\boldsymbol{w}^{k-1}$
  - Each direction $\boldsymbol{d}^p$ defines a candidate point $\boldsymbol{w}^{k-1} + \boldsymbol{d}^p$
  - The point with the smallest evaluation is chosen, i.e. the one with index: $s = \underset{p=1\ldots P}{\operatorname{argmin}}\ g\left(\boldsymbol{w}^{k-1} + \boldsymbol{d}^p\right)$

Random search

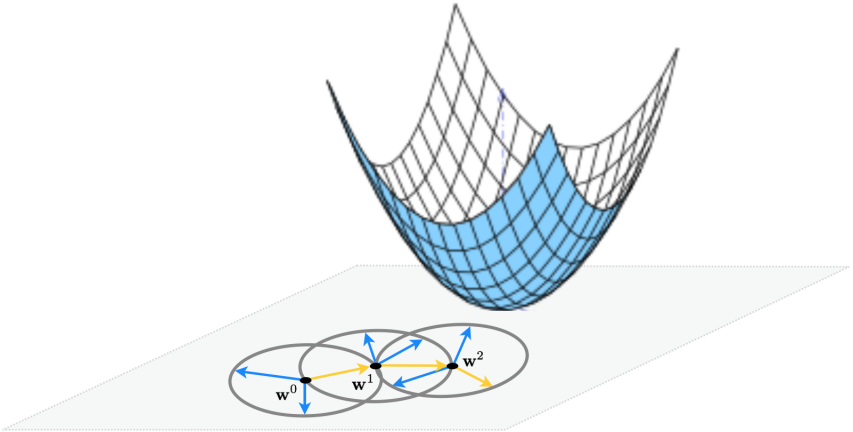
- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?
  - At step K sample a given number (say P) of directions from $\boldsymbol{w}^{k-1}$
  - Each direction $\boldsymbol{d}^p$ defines a candidate point $\boldsymbol{w}^{k-1} + \boldsymbol{d}^p$
  - The point with the smallest evaluation is chosen, i.e. the one with index: $s = \underset{p=1\ldots P}{\operatorname{argmin}}\ g\left(\boldsymbol{w}^{k-1} + \boldsymbol{d}^p\right)$
  - If $g\left(\boldsymbol{w}^{k-1} + \boldsymbol{d}^s\right) < g\left(\boldsymbol{w}^{k-1}\right)$ then we move to $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \boldsymbol{d}^s$

Random search

- A first local optimization algorithm
- Descent direction is searched by looking at a number of random directions out of current point
- Scales poorly with the dimension of the input
- Allows us to introduce some useful concepts
- How does it work?
  - At step K sample a given number (say P) of directions from $\boldsymbol{w}^{k-1}$
  - Each direction $\boldsymbol{d}^p$ defines a candidate point $\boldsymbol{w}^{k-1} + \boldsymbol{d}^p$
  - The point with the smallest evaluation is chosen, i.e. the one with index: $s = \underset{p=1\ldots P}{\operatorname{argmin}}\ g\left(\boldsymbol{w}^{k-1} + \boldsymbol{d}^p\right)$
  - If $g\left(\boldsymbol{w}^{k-1} + \boldsymbol{d}^s\right) < g\left(\boldsymbol{w}^{k-1}\right)$ then we move to $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \boldsymbol{d}^s$
  - Otherwise we halt or try another batch of P directions

Random search

- Example: a quadratic function
- $P = 3$ is the number of directions

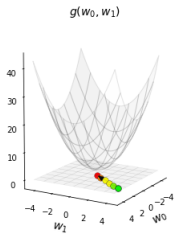


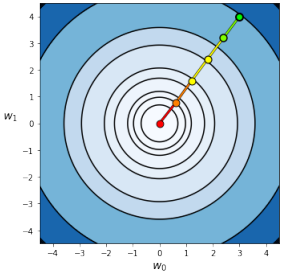
Random search

- Can we control the length of each step? Yes
- Normalize so that $\|\boldsymbol{d}\|_2 = 1$
- $\|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\|_2 = \| \left(\boldsymbol{w}^{k-1} + \boldsymbol{d}\right) - \boldsymbol{w}^{k-1}\|_2 = \|\boldsymbol{d}\|_2 = 1$
- $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}$
- $\|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\|_2 = \| \left(\boldsymbol{w}^{k-1} + \alpha\boldsymbol{d}\right) - \boldsymbol{w}^{k-1}\|_2 = \|\alpha\boldsymbol{d}\|_2 = \alpha\|\boldsymbol{d}\|_2 = \alpha$
- Example: a quadratic function
- $g(w_1, w_2) = w_1^2 + w_2^2 + 2$

Random search

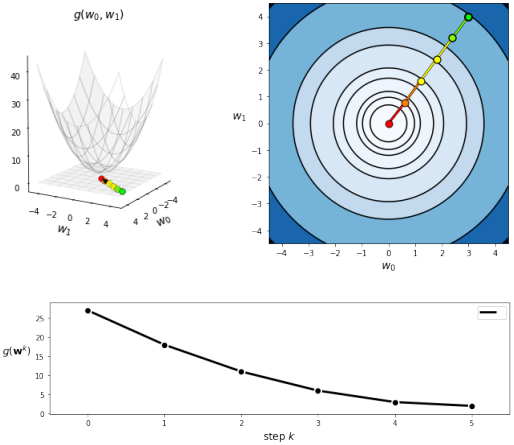
- Example: a quadratic function
- $g(w_1, w_2) = w_1^2 + w_2^2 + 2$





Random search

- Example: a quadratic function
- $g(w_1, w_2) = w_1^2 + w_2^2 + 2$



Random search

- Minimizing non convex functions may require multiple runs of local optimization
- Each with different initialisations
- Example: multiple local minima
- $g(w) = sin(3w) + 0.3w^2$

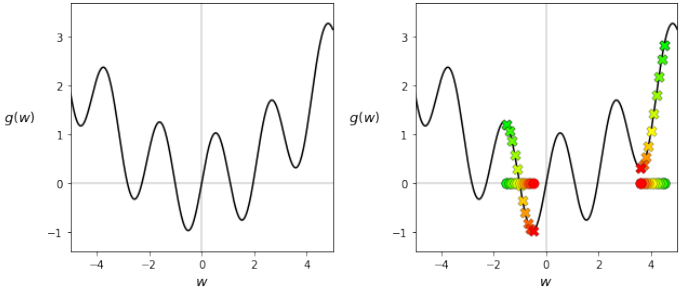
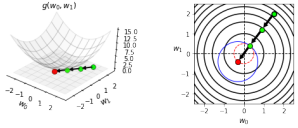


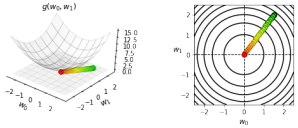
Figure: $w^0 = 4.5$ and $w^0 = -1.5$.

Random search

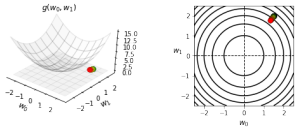
- The effect of the steplength/learning rate
- $\alpha = 1$



- $\alpha = 0.1$



- $\alpha = 0.01$



Random search

- The diminishing steplength rule:
- As an alternative to fixed α we may progressively shrink its value
- For instance, we may set $\alpha = 1/k$ at the kth step:

Random search

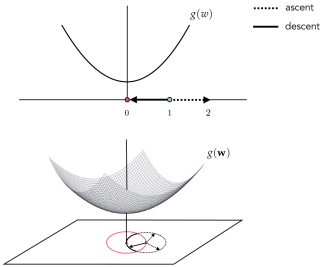
- The diminishing steplength rule:
- As an alternative to fixed α we may progressively shrink its value
- For instance, we may set $\alpha = 1/k$ at the kth step:
- $\|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\|_2 = \| \left(\boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}\right) - \boldsymbol{w}^{k-1}\|_2 = \|\alpha \boldsymbol{d}\|_2 = \alpha \|\boldsymbol{d}\|_2 = \alpha = \frac{1}{k}$

Random search

- The diminishing steplength rule:
- As an alternative to fixed α we may progressively shrink its value
- For instance, we may set $\alpha = 1/k$ at the kth step:
- $\|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\|_2 = \| \left(\boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}\right) - \boldsymbol{w}^{k-1}\|_2 = \|\alpha \boldsymbol{d}\|_2 = \alpha \|\boldsymbol{d}\|_2 = \alpha = \frac{1}{k}$
- $\sum_{k=1}^{K} \|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\|_2 = \sum_{k=1}^{K} \frac{1}{k}$

Random search

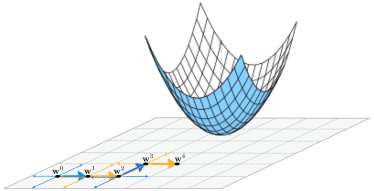
- As for global optimization, also random search suffers from the curse of dimensionality
- For most functions it is exponentially more difficult to find a descent direction at random, when input dimension grows



- For a quadratic function $g(\boldsymbol{w}) = \boldsymbol{w}^T\boldsymbol{w} + 2$ when $N = 30$ it can be demonstrated that the descent probability is $< 1\%$
- Very much inefficient
- Can we get around?

Coordinate search and descent

- The concept: while random search minimize functions w.r.t. all coordinates simultaneously, *coordinate-wise* algorithms try to minimize with respect to one (or one subset of) coordinate(s) at the time



Coordinate search and descent

- An example:

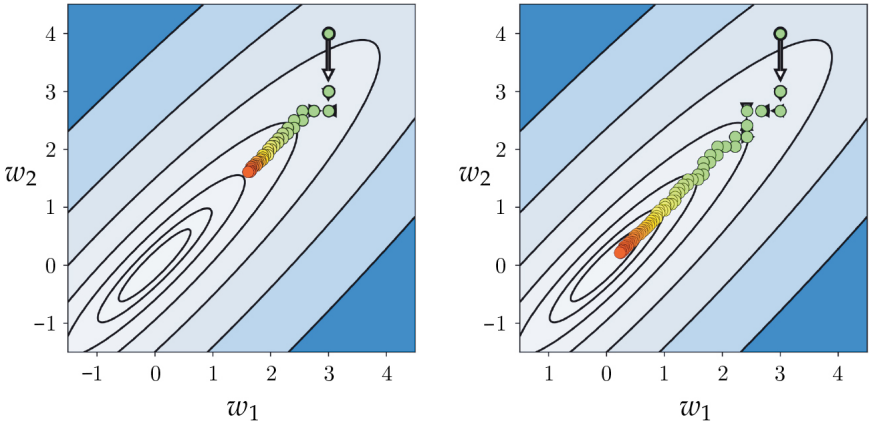
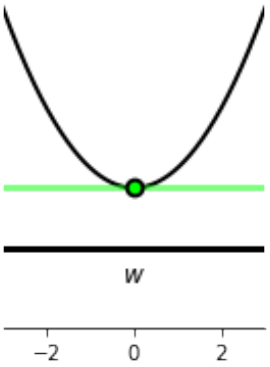
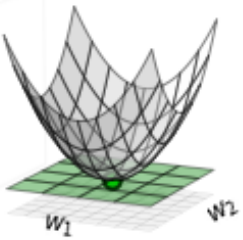


Figure: Coordinate search (a) and descent (b).

First-order optimization techniques

- First-order techniques rely on *first derivative* (or *gradient*) of a function
- Remark: for differentiable functions, tangent lines/hyperplanes are perfectly *flat* at minimum points





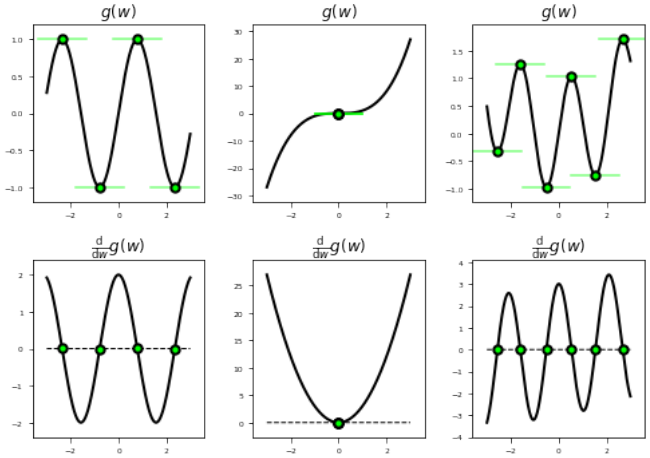
The first-order optimality condition

- $\frac{\mathrm{d}}{\mathrm{d}w} g(v) = 0$
- $\begin{array}{c} \frac{\partial}{\partial w_1} g(\boldsymbol{v}) = 0 \\ \frac{\partial}{\partial w_2} g(\boldsymbol{v}) = 0 \\ \vdots \\ \frac{\partial}{\partial w_N} g(\boldsymbol{v}) = 0 \end{array}$
- $\nabla g(\boldsymbol{v}) = 0_{N \times 1}$

- Solving this system of equations in closed (algebraic) form by hand is very difficult
- First-order condition corresponds to global minima for *convex* functions (e.g. quadratic ones as in Figure)
- In general, it corresponds to a condition met by *stationary* or *critical* points: minima, maxima, saddle

First-order optimization

- In general, it corresponds to a condition met by *stationary* or *critical* points



The first-order optimality condition

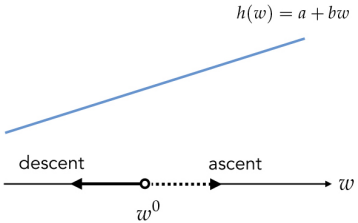
- $$\begin{aligned} \frac{\partial}{\partial w_1} g(\boldsymbol{v}) &= 0 \\ \frac{\partial}{\partial w_2} g(\boldsymbol{v}) &= 0 \\ &\vdots \\ \frac{\partial}{\partial w_N} g(\boldsymbol{v}) &= 0 \end{aligned}$$
- Solving this system of equations in closed (algebraic) form by hand is very difficult
- But sometimes can be solved one equation at the time (i.e. sequentially)
- This is a sort of coordinate descent idea, effective especially if each of the equation can be solved in closed form (e.g. when the function to be minimized is quadratic)

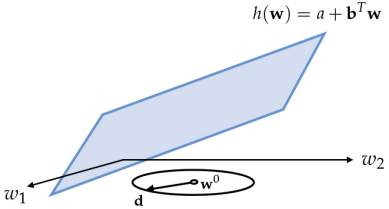
First-order Taylor series

- A geometric view
- A general N-dimensional hyperplane: $h(w_1, w_2, ..., w_N) = a + b_1 w_1 + b_2 w_2 + \cdots b_N w_N$
- $h(\boldsymbol{w}) = a + \boldsymbol{b}^T \boldsymbol{w}$
- With $[\boldsymbol{b}] = n \times 1$ and $[\boldsymbol{w}] = n \times 1$
- If $N = 1$, we have a one-dimensional object. Its input space (defined by w) is one-dimensional. It *lives* in two-dimensional space.
- For a generic N, we have a N-dimensional object. Its input space (defined by w) is N-dimensional. It *lives* in a (N+1)-dimensional space.
- When $N = 1$ is easy to discern, at a point w_0, if we increase or decrease the value of the function moving away from w_0: we only have 2 directions
- But what for $N > 1$?
- Which is, among the infinite directions the one that produces the largest/steepest ascent (or descent)?

First-order Taylor series

- Which is, among the infinite directions the one that produces the largest/steepest ascent (or descent)?
- At point $\boldsymbol{w}^0$ we aim at finding the unit direction $\boldsymbol{d}$ s.t.: $h(\boldsymbol{w}^0 + d)$ is maximal
- $\underset{\boldsymbol{d}}{\text{maximize}} \;\; h\left(\boldsymbol{w}^0 + d\right)$
- Given the expression for h, we may write:
- $a + \boldsymbol{b}^T(\boldsymbol{w}^0 + \boldsymbol{d}) = a + \boldsymbol{b}^T\boldsymbol{w}^0 + \boldsymbol{b}^T\boldsymbol{d}$





First-order Taylor series

- Hence, maximizing w.r.t $\boldsymbol{d}$ is equivalent to maximize $\boldsymbol{b}^T\boldsymbol{d} = \|b\|_2\|d\|_2 cos(\theta)$
- Since $\|b\|_2$ doesn't depend for $\boldsymbol{d}$ and $\|\boldsymbol{d}\|_2 = 1$, this means:
- $\underset{\boldsymbol{\theta}}{\text{maximize}} \;\; cos(\theta)$
- $\boldsymbol{d} = \frac{\boldsymbol{b}}{\|\boldsymbol{b}\|_2}$ with $\theta = 0$, $cos(\theta) = 1$ provides the steepest ascent
- $\boldsymbol{d} = \frac{-\boldsymbol{b}}{\|\boldsymbol{b}\|_2}$ with $\theta = \pi$, $cos(\theta) = -1$ provides the steepest descent

Gradient and steepest ascent/descent

- A multiple variable function can be (locally, around $\boldsymbol{w}^0$) approximated by a hyperplane $h(\boldsymbol{w})$
- $h(\boldsymbol{w}) = g(\boldsymbol{w}^0) + \nabla g(\boldsymbol{w}^0)^T(\boldsymbol{w} - \boldsymbol{w}^0) = a + \boldsymbol{b}^T\boldsymbol{w}$
- h is the first-order Taylor series approximation of g at $\boldsymbol{w}^0$, and is tangent to g at $\boldsymbol{w}^0$
- Since h closely approximates g near $\boldsymbol{w}^0$ we may use its steepest ascent/descent directions to increase/decrease g near $\boldsymbol{w}^0$

Gradient descent

- The gradient $-\nabla g(\boldsymbol{w})$ of $g(\boldsymbol{w})$ always defines a valid descent direction
- What about using a local optimization scheme of the type:
  - $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}^k$ with negative gradient direction $\boldsymbol{d}^k = -\nabla g\left(\boldsymbol{w}^{k-1}\right)$
  - Leading to: $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} - \alpha \nabla g\left(\boldsymbol{w}^{k-1}\right)$
- This is known as the *gradient descent algorithm*

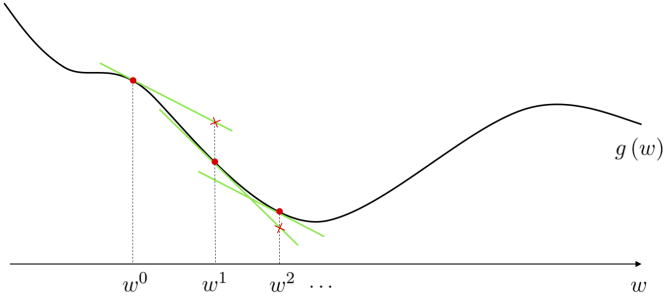
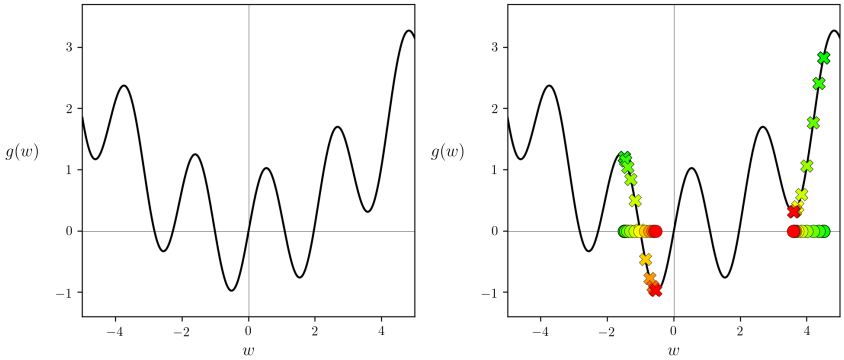


Figure: The gradient descent algorithm at work on a single-input function

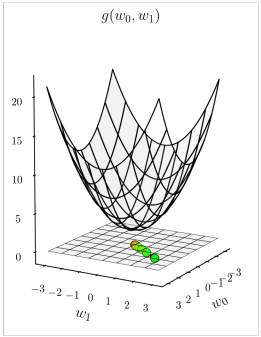
Gradient descent

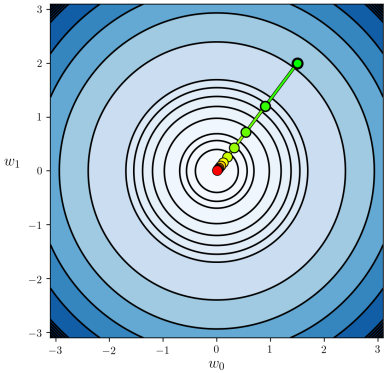
- Example: minimization of a nonconvex function



Gradient descent

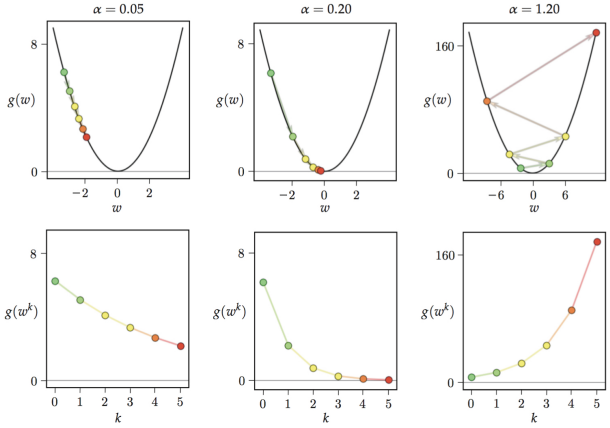
- Example: minimization of a convex function





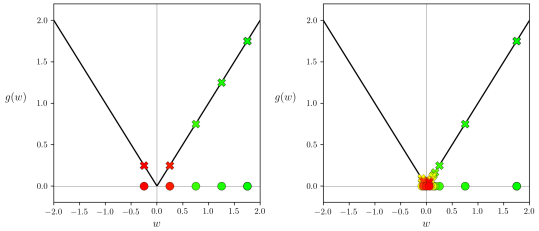
Gradient descent

- The steplength choice
- Fixed value, usually $\alpha = 10^{\gamma}$, with $\gamma \in \mathbb{Z}$ and usually < 0
- Diminishing steplength, like $\alpha = \frac{1}{k}$ at kth iteration

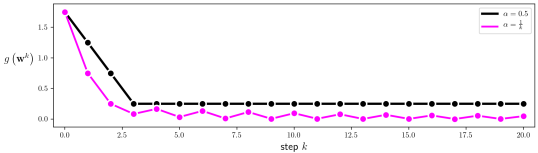


Gradient descent

- The steplength choice
- Fixed value, usually $\alpha = 10^{\gamma}$, with $\gamma \in \mathbb{Z}$ and usually < 0
- Diminishing steplength, like $\alpha = \frac{1}{k}$ at kth iteration



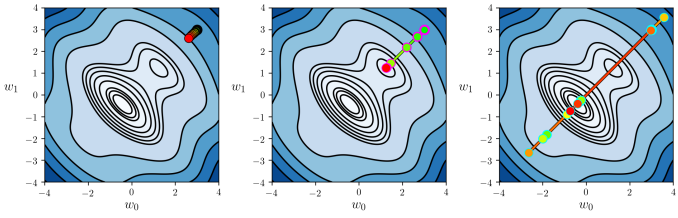
(a)



(b)

Gradient descent

- An example of oscillation



Gradient descent

- Convergence criteria: when do we stop the algorithm?
- If the step from $\boldsymbol{w}^{k-1}$ to $\boldsymbol{w}^{k}$ doesn't lead to significant improvement
- i.e. $-\nabla g\left(\boldsymbol{w}^{k}\right) \approx 0_{N\times 1}$ (is vanishing) and we are next to a stationary point
- Formally we may wait until $\left\|\nabla g\left(\boldsymbol{w}^{k-1}\right)\right\|_2 < \epsilon$, or $\frac{1}{N}\|\boldsymbol{w}^{k} - \boldsymbol{w}^{k-1}\|_2 < \epsilon$, or $\frac{1}{N}|g(\boldsymbol{w}^{k}) - g(\boldsymbol{w}^{k-1})| < \epsilon$
- From a practical point of view we may also fix a maximum iteration count
  - Depending on function, computing resources, choice of α
  - Small values of α may lead to poor progress
  - Large values of α may lead to bouncing around

Gradient descent

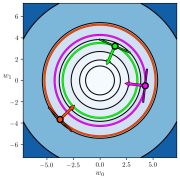
- Two shortcomings of the method
- The gradient is a vector, characterized by *direction* and *magnitude*
- Depending on the function being minimized, the direction of the negative gradient may rapidly oscillate, leading to slow convergence (zig-zag steps)
- The *magnitude* may vanish rapidly near stationary points, slowing down the algorithm progress near these points (slow-crawling)
- This is true for instance if functions to minimise have long *narrow valleys* (i.e. long flat areas with contour increasingly parallel)

Gradient descent

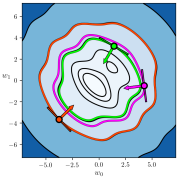
Property

The gradient ascent/descent direction of any differentiable function at all its inputs $\boldsymbol{w}^0$ is always perpendicular to the contour $g(\boldsymbol{w}) = g(\boldsymbol{w}^0)$

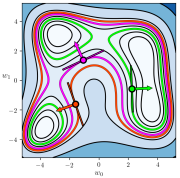
- Examples:



(a)



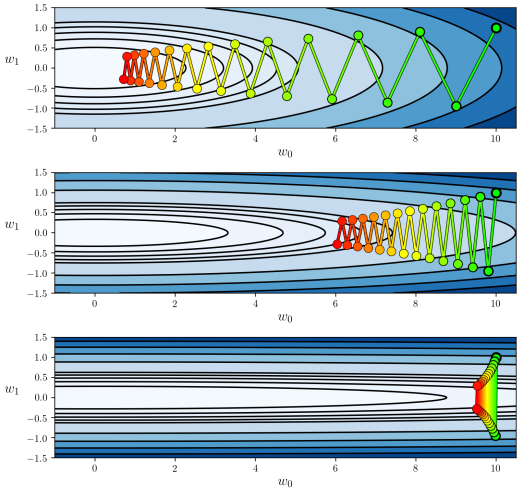
(b)



(c)

Gradient descent

- Example of zig-zagging behavior:

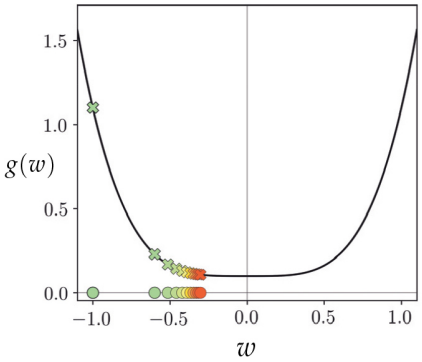


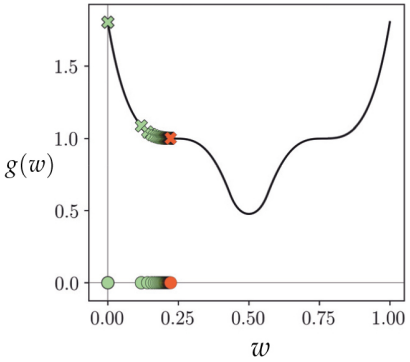
Gradient descent

- Slow-crawling behavior
- At stationary points $\nabla g(\boldsymbol{w}) = 0$
- Hence, also $\|\nabla g(\boldsymbol{w}) = 0\|_2$ holds
- Meaning $\nabla g(\boldsymbol{w}) \approx 0$ near stationary points (non-zero direction but vanishing magnitude)
- Distance "traveled" by a gradient descent step:
  $\left\|\boldsymbol{w}^k - \boldsymbol{w}^{k-1}\right\|_2 = \left\|\left(\boldsymbol{w}^{k-1} - \alpha \nabla g\left(\boldsymbol{w}^{k-1}\right)\right) - \boldsymbol{w}^{k-1}\right\|_2 = \alpha \left\|\nabla g\left(\boldsymbol{w}^{k-1}\right)\right\|_2$
- Which is α times the magnitude of the gradient direction
- As a consequence, first steps (often random intialization) can make large progress
- Small progress is made when approaching stationary points

Gradient descent

- Example of slow-crawling behavior:



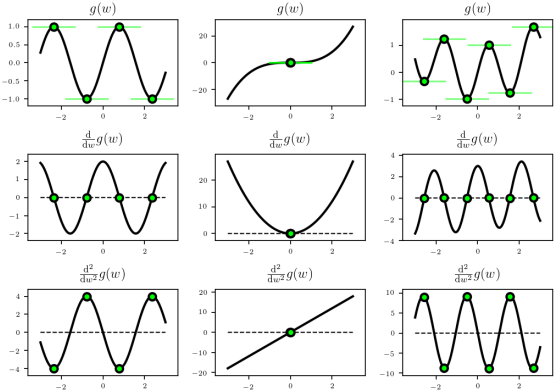


Second-order optimization techniques

- To determine if a single-input function $g(w)$ is convex/concave at a point v we may study its curvature
- If $\frac{\mathrm{d}^2}{\mathrm{d}w^2}g(v) \geq 0$ $(\leq 0) \rightarrow g$ is convex (concave)
- If g has multi-dimensional input, we analogously study the Hessian matrix
- The Hessian $\nabla^2 g(\boldsymbol{v})$(of a twice continuous differentiable) function $g(\boldsymbol{v}) : \mathbb{R}^N \rightarrow \mathbb{R}$ is an $N \times N$ matrix that collects all second-order partial derivatives of g, which measures the curvature of the function locally around $\boldsymbol{v}$
- If $g(\boldsymbol{v}) : \mathbb{R}^N \rightarrow \mathbb{R}^M$ (i.e. it's a vector field) the Hessian can still be defined (as a $M \times N \times N$ tensor)
- If $\nabla^2 g(\boldsymbol{v})$ has all nonnegative (nonpositive) *eigenvalues* then g is said to be convex (concave) at $\boldsymbol{v}$ (and the Hessian matrix is said to be positive (negative) semi-definite)

Second-order optimization techniques

- Second-order definition of convexity
  - $g(w)$ is convex *everywhere* if $\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v)$ is always ≥ 0
  - $g(\boldsymbol{v})$ is convex *everywhere* if $\nabla^2 g(\boldsymbol{w})$ always has ≥ 0 eigenvalues
- Example:



Second-order optimization techniques

- The examples give a picture of how stationary points relate to second derivatives and can be generalized as follows:

Second-order optimality condition

A stationary point v is:

- a local (global) minimum if $\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v) > 0$ (it occurs at convex portions of the function)
- a local (global) maximum if $\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v) < 0$ (it occurs at concave portions of the function)
- a saddle point if $\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v) = 0$ and if $\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v)$ changes sign at $w = v$ (it occurs at an inflection point, from concave to convex or vice versa)

- Analogous second-order conditions hold for multiple-input functions:

Second-order optimality condition

A stationary point v of a multi-input function $g(\boldsymbol{w})$ is:

- a local (global) minimum if all eigenvalues of $\nabla^2 g(\boldsymbol{v})$ are positive (it occurs at convex portions of the function)
- a local (global) maximum if all eigenvalues of $\nabla^2 g(\boldsymbol{v})$ are negative (it occurs at concave portions of the function)
- a saddle point if the eigenvalues of $\nabla^2 g(\boldsymbol{v})$ are mixed (i.e. some > 0 and some < 0 (it occurs at an inflection point, from concave to convex or vice versa)

Second-order Taylor series

- A geometric view
- Single-input quadratic functions: $g(w) = a + bw + cw^2$
  - c controls concavity/convexity
- Multi-input quadratic functions: $g(\boldsymbol{w}) = a + \boldsymbol{b}^T\boldsymbol{w} + \boldsymbol{w}^T\boldsymbol{C}\boldsymbol{w}$
  - C is a (symmetric) $N \times N$ matrix. If its eigenvalues are all ≥ 0, g is convex, if they are all ≤ 0 g is concave. Otherwise neither convex nor concave.
- A geometric view
- Second-order Taylor series and local curvature
- Function g can be locally approximated at point v as:
  - $h(w) = g(v) + (\frac{\mathrm{d}}{\mathrm{d}w}g(v))(w - v) + \frac{1}{2}(\frac{\mathrm{d}^2}{\mathrm{d}w^2}g(v))(w - v)^2$
- If the function is convex/concave at v, the approximating quadratic (i.e. the 2nd order Taylor series h) is convex/concave everywhere

Second-order Taylor series

- A geometric view
- Second-order Taylor series and local curvature for multi-input functions
- Function g can be locally approximated at point $\boldsymbol{v}$ as:
  - $h(\boldsymbol{w}) = g(\boldsymbol{v}) + \nabla g(\boldsymbol{v})^T(\boldsymbol{w} - \boldsymbol{v}) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{v})^T \nabla^2 g(\boldsymbol{v})(\boldsymbol{w} - \boldsymbol{v})$
- If the function g is convex/concave at v, the approximating quadratic (i.e. the 2nd order Taylor series h) is convex/concave everywhere

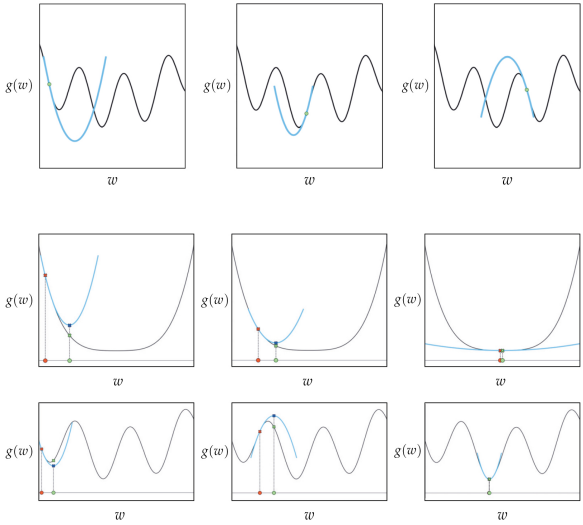
Newton's method

- First-order Taylor series approximation leads to local optimization of gradient descent
- Do higher-order Taylor series lead to descent-based algorithms? Yes
- Newton's method is particularly useful for minimization of convex functions with moderate number of inputs
- Using the 1st order condition to solve for stationary point $w^\star$ of the quadratic representing the 2nd-order Taylor approximation, setting its derivative to zero leads to:
  - $w^\star = v - \frac{\frac{\mathrm{d}}{\mathrm{d}w} g(v)}{\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(v)}$
- Analogously, for multi-input functions we may set the gradient of the quadratic approximation to zero and solve to get the stationary point $\boldsymbol{w}^\star$:
  - $\boldsymbol{w}^\star = \boldsymbol{v} - (\nabla^2 g(\boldsymbol{v}))^{-1} \nabla g(\boldsymbol{v})$
- To get to stationary point $w^\star$ we may move from $\boldsymbol{v}$ in the direction defined by $(\nabla^2 g(\boldsymbol{v}))^{-1} \nabla g(\boldsymbol{v})$

Newton's method

- Is the direction by $(\nabla^2 g(\boldsymbol{v}))^{-1} \nabla g(\boldsymbol{v})$ a descent direction?
- In case the function is convex everywhere, the quadratic approximation not only matches the curvature but is always convex $\Rightarrow$ its stationary point is a global minimum
- Moreover, the minimum $\boldsymbol{w}^\star$ of the quadratic function leads to lower points on the function, i.e. $g(\boldsymbol{w}^\star) < g(\boldsymbol{v})$
- The key intuition behind the Newton's method is therefore to repeatedly travel to points defined by stationary points of the second-order Taylor approximation
- For convex functions this leads to the minimization of the function

Newton's method



Newton's method

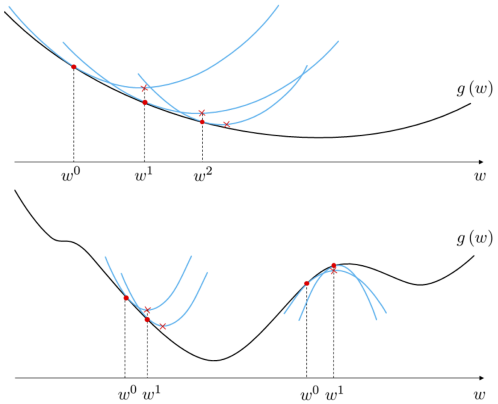
- The algorithm repeatedly takes steps to stationary points of the second-order Taylor approximation:
- For a single-input function, at kth step it computes a second-order Taylor series approximation centered at w^{k-1}:
  - $h(w) = g(w^{k-1}) + \left(\frac{\mathrm{d}}{\mathrm{d}w} g(w^{k-1})\right)(w - w^{k-1}) + \frac{1}{2}\left(\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(w^{k-1})\right)(w - w^{k-1})^2$
- Then, it solves for its stationary point to compute w^k as:
  - $w^k = w^{k-1} - \frac{\frac{\mathrm{d}}{\mathrm{d}w} g(w^{k-1})}{\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(w^{k-1})}$
- For multi-input functions at kth iteration the algorithm computes the 2nd order quadratic approximation:
  - $h(\boldsymbol{w}) = g(\boldsymbol{w}^{k-1}) + \nabla g(\boldsymbol{w}^{k-1})^T(\boldsymbol{w} - \boldsymbol{w}^{k-1}) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^{k-1})^T \nabla^2 g\left(\boldsymbol{w}^{k-1}\right)(\boldsymbol{w} - \boldsymbol{w}^{k-1})$
- and...

Newton's method

- ...and solves for a stationary point of this approximation, computing $\boldsymbol{w}^k$:
  - $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} - \left(\nabla^2 g(\boldsymbol{w}^{k-1})\right)^{-1} \nabla g(\boldsymbol{w}^{k-1})$
- Notice that this local optimization algorithm belongs to the class following the general form update: $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} + \alpha \boldsymbol{d}^k$, where $\boldsymbol{d}^k = -\left(\nabla^2 g(\boldsymbol{w}^{k-1})\right)^{-1} \nabla g(\boldsymbol{w}^{k-1})$ and $\alpha = 1$
- Problem: we need to solve $\nabla^2 g(\boldsymbol{w}^{k-1})^{-1}$, that means inverting the $N \times N$ Hessian matrix (computationally expensive)
- It can be proved that $\boldsymbol{w}^k$ can be found equivalently (and in practice in a more efficient way) by solving an equivalent symmetric system of equations:
  $\nabla^2 g\left(\boldsymbol{w}^{k-1}\right) \boldsymbol{w} = \nabla^2 g\left(\boldsymbol{w}^{k-1}\right) \boldsymbol{w}^{k-1} - \nabla g(\boldsymbol{w}^{k-1})$

Newton's method

- Example of algorithm at work:



- Generally Newton's method requires fewer steps for convergence w.r.t. gradient descent, but it's more difficult to use with nonconvex functions (at concave portions it can climb to a local maximum)

Newton's method

- Example of algorithm at work: $g(w_1, w_2) = 0.26(w_1^2 + w_2^2) - 0.48 w_1 w_2$

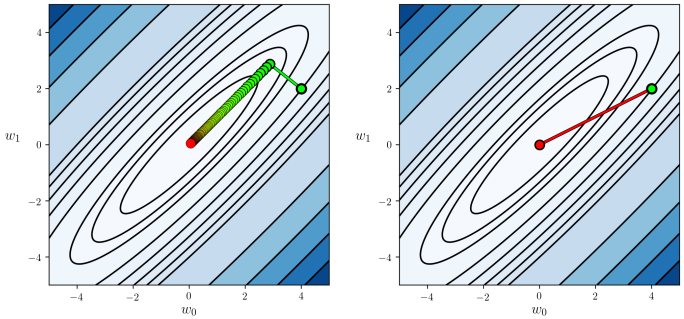


Figure: Gradient descent algorithm (a) and Newton's method (b)

Newton's method

- $w^k = w^{k-1} - \frac{\frac{\mathrm{d}}{\mathrm{d}w} g(w^{k-1})}{\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(w^{k-1})}$
- Numerical stability issues: near flat portions of g, the numerator and denominator have near-zero values
- To avoid this problem (especially division by zero) a *regularized* Newton's step can be used: $w^k = w^{k-1} - \frac{\frac{\mathrm{d}}{\mathrm{d}w} g(w^{k-1})}{\frac{\mathrm{d}^2}{\mathrm{d}w^2} g(w^{k-1}) + \epsilon}$
- ϵ is typically small (e.g. 10^{-7})
- The regularized Newton's step version for multi-input functions is: $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} - \left(\nabla^2 g(\boldsymbol{w}^{k-1}) + \epsilon \boldsymbol{I}_{N \times N}\right)^{-1} \nabla g(\boldsymbol{w}^{k-1})$
- Steplength choice: in standard Newton's algorithm $\alpha = 1$ is implicitly assumed. However a version that accounts for steplength value can be introduced: $\boldsymbol{w}^k = \boldsymbol{w}^{k-1} - \alpha \left(\nabla^2 g(\boldsymbol{w}^{k-1})\right)^{-1} \nabla g(\boldsymbol{w}^{k-1})$

Newton's method

- Historical note: Newton's method was invented as a zero-finding algorithm (not as a local optimization algorithm)
- Pros and cons of the algorithm:
  - Since first-order and second-order (curvature) information are used, it doesn't suffer "zig-zag" issues of first-order algorithms
  - Quadratic approximation enables fast convergence (especially close to a minimum)
  - However, the single step is more expensive than a first-order step
  - Storing the Hessian matrix can be challenging for high-dimensional data (if $N = 10^4$, the hessian matrix has 10^8 entries)
  - When applied to nonconvex functions it can in principle take steps uphill
  - Some countermeasures could in principle be applied to ameliorate these two last issues